BATCH 26

Inceptez Data Science Presents

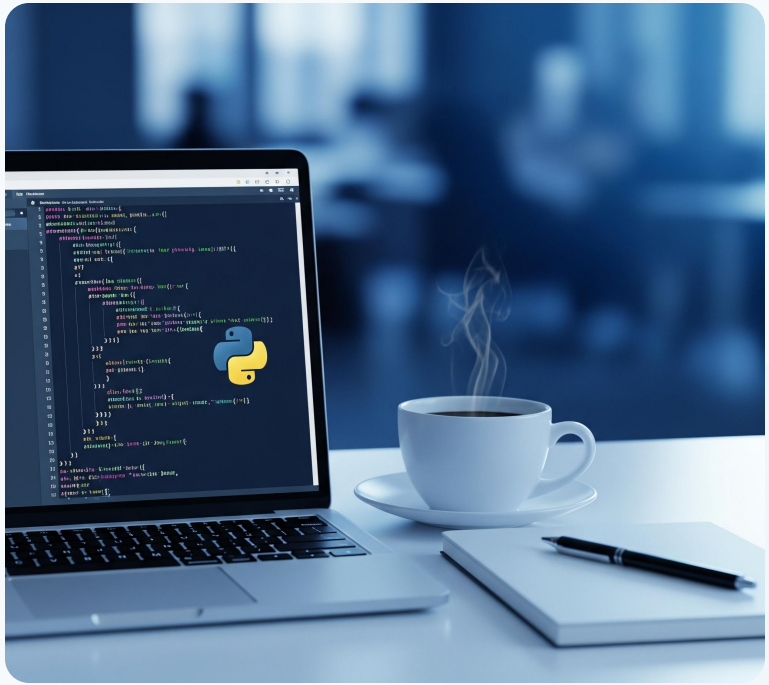
Python Day - 1

Laxmi Narayen Nagarajan Venkatesan

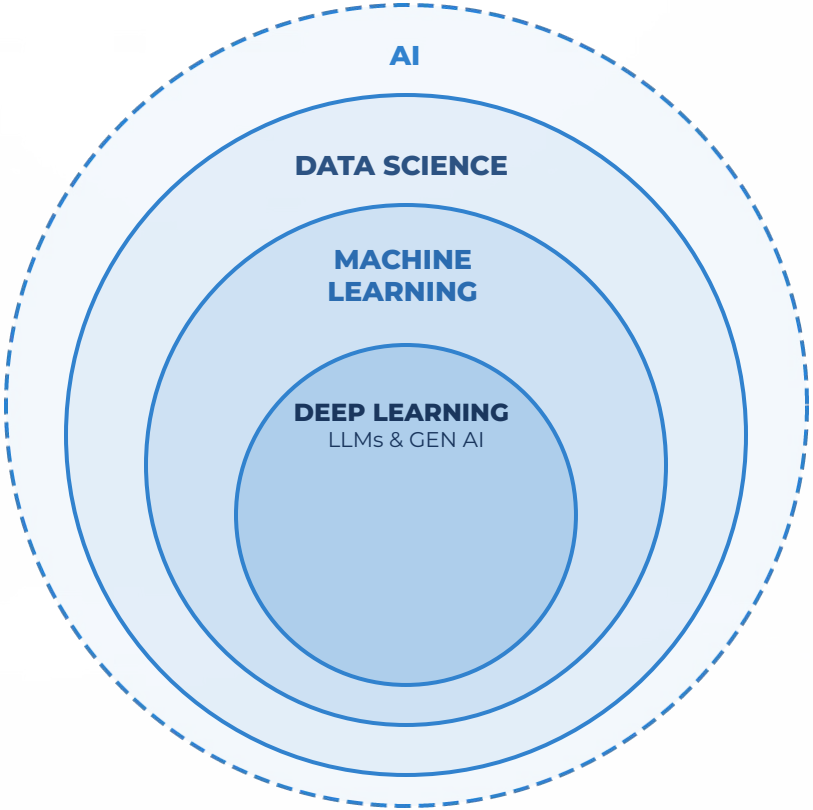
18th-April-2026

REVIEW

Reflecting on Our Last Sessions



The AI Hierarchy & Evolution



Data Science

The overarching field using statistics, domain expertise, and math to extract actionable insights from complex datasets.

Machine Learning

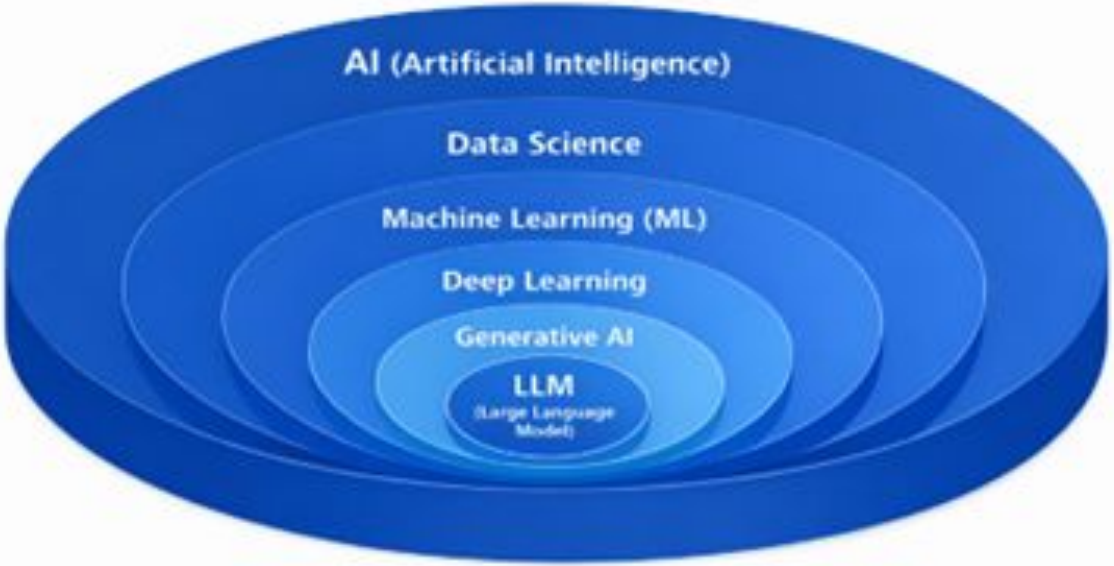
A subset of AI using algorithms like Regression & Trees that improve through experience and data patterns.

Deep Learning & NLP

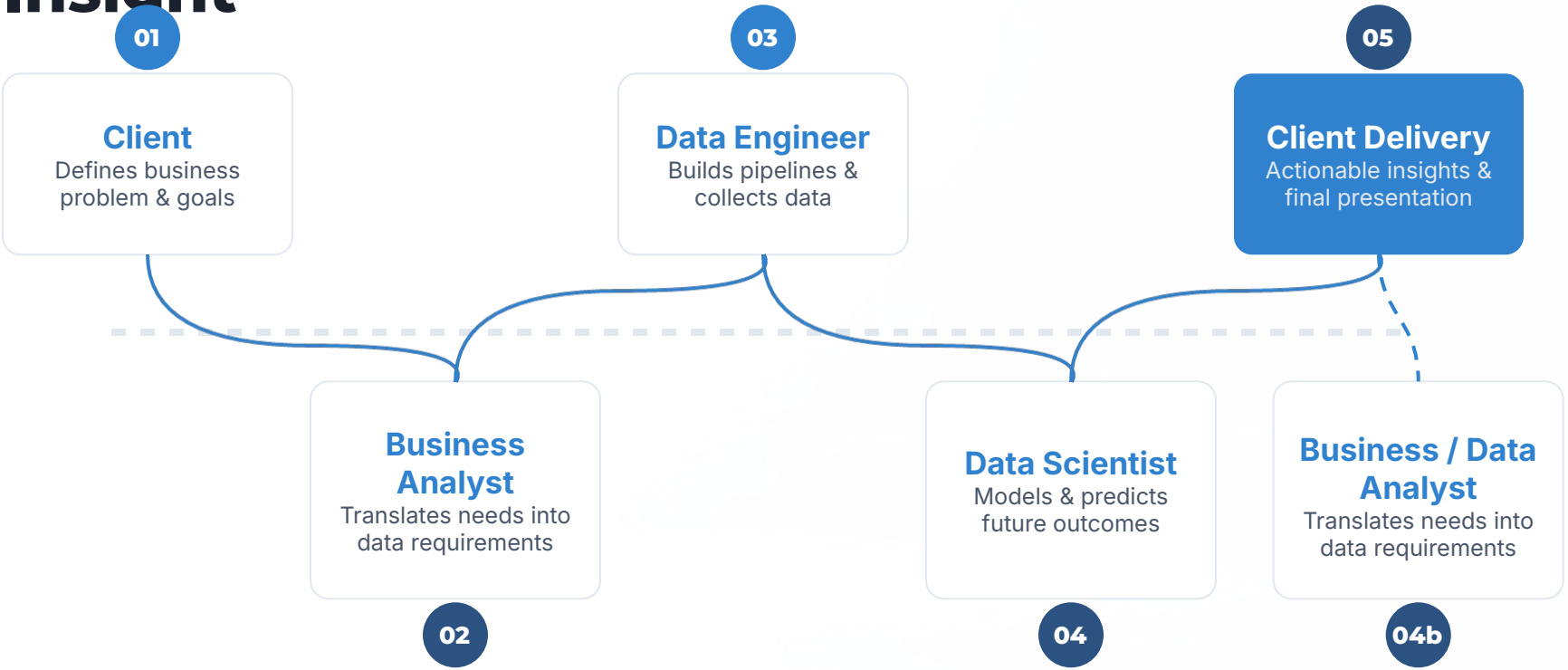
Neural Networks (ANN/CNN/RNN) that mimic human cognitive functions to process complex data like text and images.

LLMs & Gen-AI

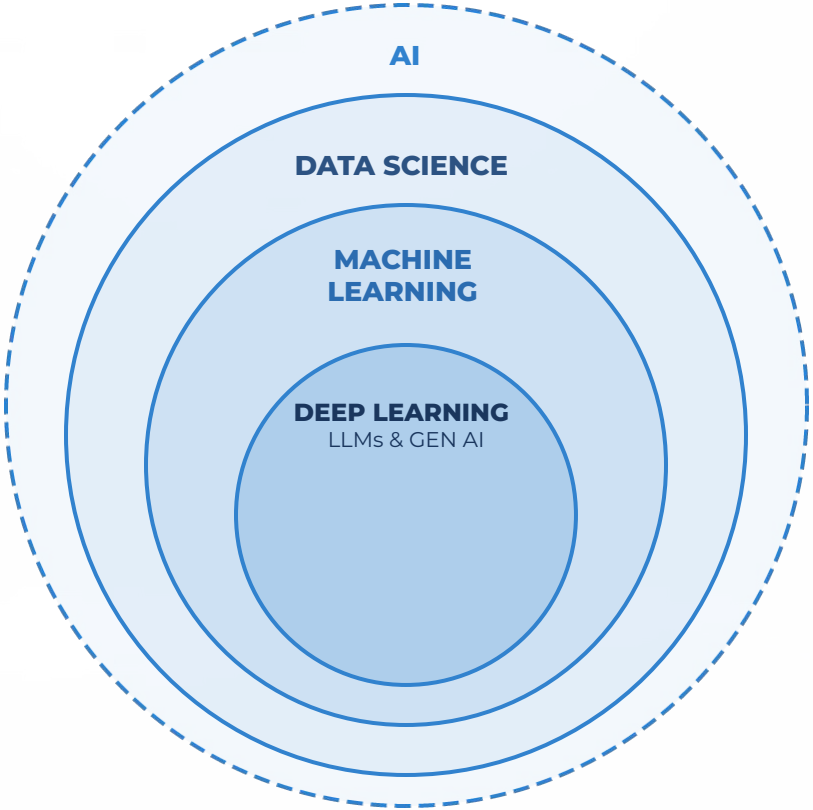
Massive Transformer-based models (GPT, Llama) representing the latest evolution in autonomous decision-making.


AI (Artificial Intelligence)
Data Science
Machine Learning (ML)
Deep Learning
Generative AI
LLM
(Large Language Model)

Data Project Lifecycle: From Client to Insight



The AI Hierarchy & Evolution



Data Science

The overarching field using statistics, domain expertise, and math to extract actionable insights from complex datasets.

Machine Learning

A subset of AI using algorithms like Regression & Trees that improve through experience and data patterns.

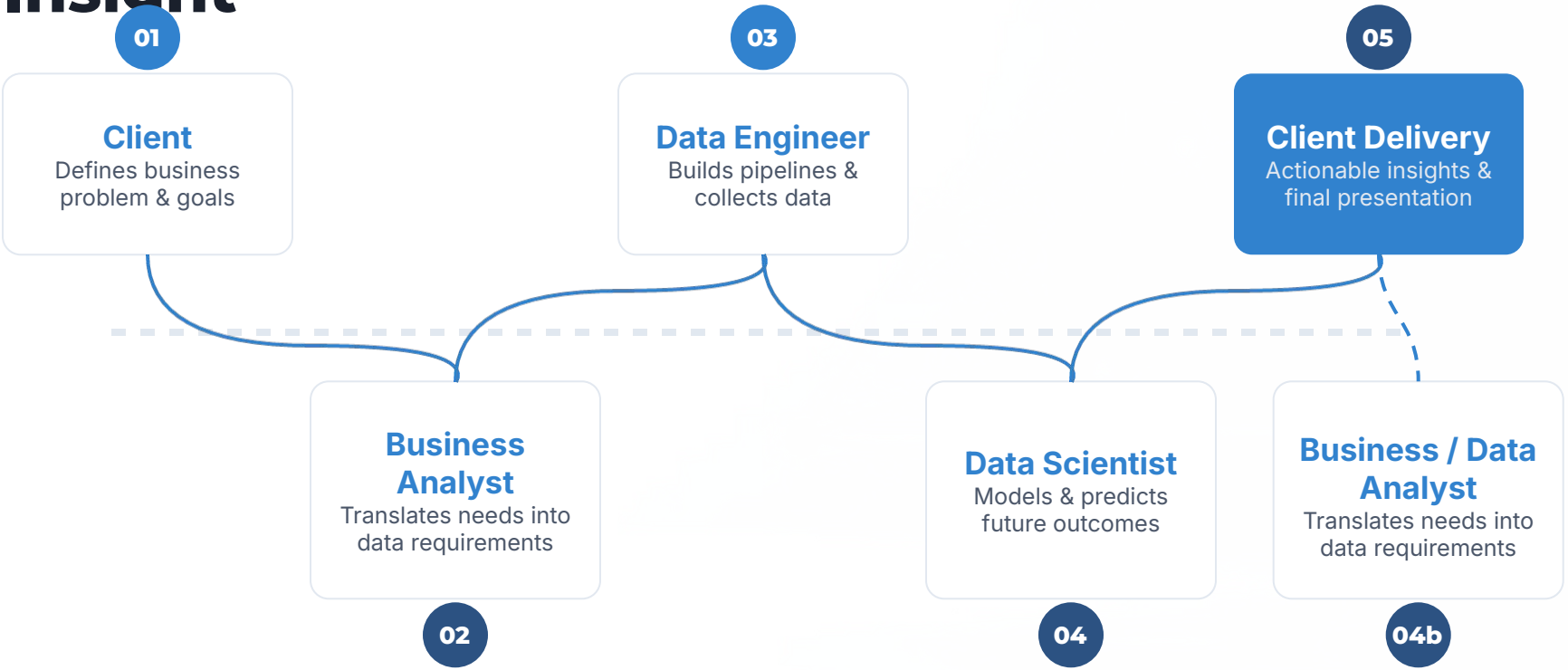
Deep Learning & NLP

Neural Networks (ANN/CNN/RNN) that mimic human cognitive functions to process complex data like text and images.

LLMs & Gen-AI

Massive Transformer-based models (GPT, Llama) representing the latest evolution in autonomous decision-making.

Data Project Lifecycle: From Client to Insight



Rapid Prototyping & Agentic Workflows

Zero-Code Data Science is possible

LIVE DEMO

End-to-end deployment of a data collection and analysis engine using natural language instructions.

Data Collection & CSV Integration

Automated ingestion of user data via CSV, structured and cleaned by agentic logic without manual coding.

Descriptive & Deep Analysis

Instant generation of statistical profiles, descriptive summaries, and behavioral insights on live datasets.

Descriptive & Deep Analysis

Instant generation of statistical profiles, descriptive summaries, and behavioral insights on live datasets.

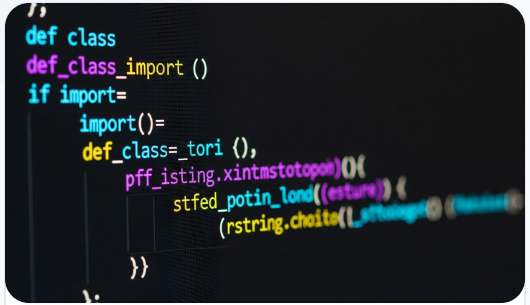
100%
No-Code Pipeline

Real-Time
Analysis & Profiling

Public
Streamlit Cloud Hosting

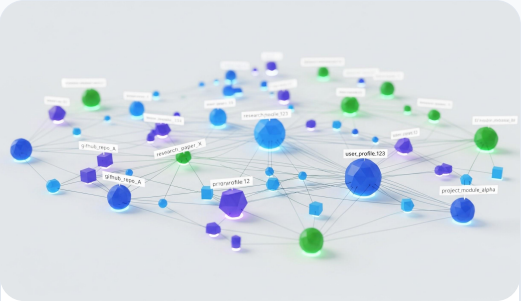
Foundational Excellence in Data Science

Why Code Still Rules in a No-Code scene?



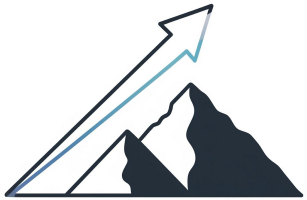
1. Customization & Debugging

- **Precise Error Solving:** Move beyond vague no-code errors to pinpoint issues in the underlying framework.
- **Custom Logic:** Build unique transformations and visualizations that predefined tools can't support.



2. Research & Innovation

- **GitHub Fluency:** Access cutting-edge ML models released by research labs before they hit no-code tools.
- **Adapt & Replicate:** Modify state-of-the-art research papers to solve specific business problems.



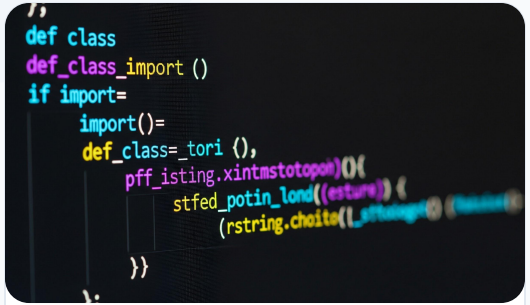
3. Flexibility & Growth

- **Scalable Data:** Use code to handle massive datasets and cloud infra where UI tools struggle.
- **Automation:** Deploy robust production pipelines, differentiating yourself from basic analysts.

Foundational Excellence in Data Science

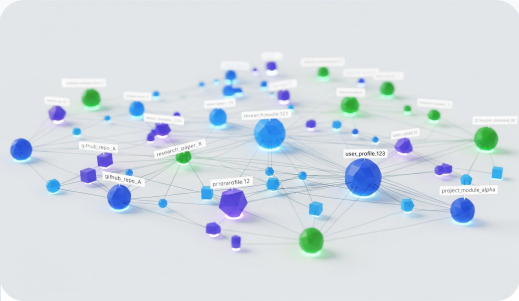
Why Code Still Rules in a No-Code scene?

Compliance



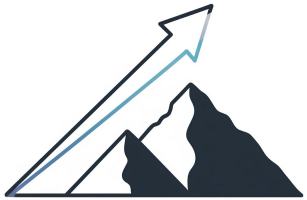
1. Customization & Debugging

- **Precise Error Solving:** Move beyond vague no-code errors to pinpoint issues in the underlying framework.
- **Custom Logic:** Build unique transformations and visualizations that predefined tools can't support.



2. Research & Innovation

- **GitHub Fluency:** Access cutting-edge ML models released by research labs before they hit no-code tools.
- **Adapt & Replicate:** Modify state-of-the-art research papers to solve specific business problems.



3. Flexibility & Growth

- **Scalable Data:** Use code to handle massive datasets and cloud infra where UI tools struggle.
- **Automation:** Deploy robust production pipelines, differentiating yourself from basic analysts.

Note: There are strict compliances and regulations where sometimes data cannot be sent outside.

PART 1

What is Python? (Beyond the program)

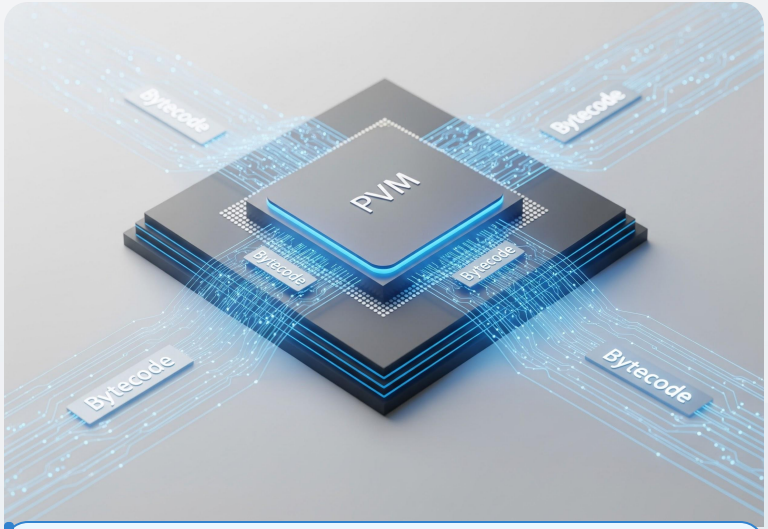
The Execution Model

High-Level & Interpreted: Not compiled to machine code like C++; it's platform-independent.

The Workflow:

- Source Code (.py) → Bytecode (.pyc)
- Bytecode processed via **PVM (Python Virtual Machine)**
- Note: Python does **not** use JVM by default.

Memory Management: Automatic Garbage Collection and Reference Counting—crucial for large AI datasets.



💡 Interview Tip

"CPython" is the standard. Use **Jython** for JVM, but stick to **CPython** for AI/Data Science to ensure C-extension support (NumPy/Pandas).

PART 2

Ways to Run Python in 2026

CLI - codes

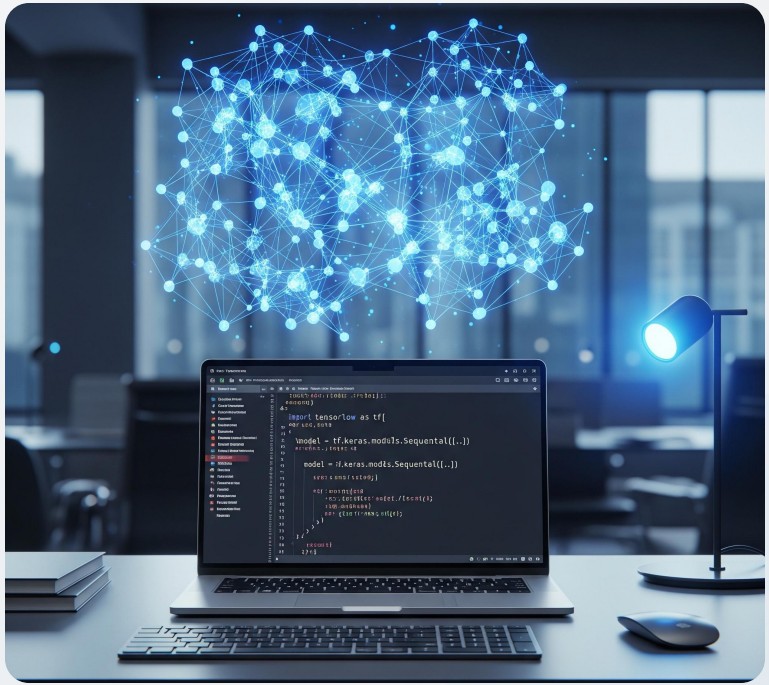
Terminal: Python - Python Terminal

Interactive Notebooks - IDEs

Jupyter & Colab: Leveraging kernels for rapid prototyping, rich visualization, and collaborative cloud computing.

Python scripts

Either use partitioned virtual Envs / Base envs to write python scripts



BASICS

Python Syntax: The Rules of the Road

What is Syntax?

Syntax refers to the **set of rules** that defines the combinations of symbols that are considered to be a correctly structured program in Python. It's the "grammar" of the code.

The Cost of Non-Compliance

If you do not strictly adhere to the syntax, Python will raise a **SyntaxError** and the program will **refuse to run**.

Common Triggers:

- Missing colons (`:`) after statements.
- Incorrect indentation (crucial in Python).
- Unclosed parentheses or quotes.

PART 3

The Building Blocks: Keywords & Variables

1. Keywords

Python's "locked" vocabulary. These are the **verbs** of the language.

- **Fixed Meaning:** Tells Python exactly what logic to run.
- **The Golden Rule:** Strictly forbidden for naming data (e.g., no variables named `if` or `for`).

2. Identifiers

The names you create for variables or projects.

✅ **DO:** Start with letters or `_`. Be case-sensitive (`Data` ≠ `data`).

❌ **DON'T:** Start with numbers, use spaces, or special characters like `@` or `%`.

3. Variables: Labeled Containers

```
course_duration = 10
VALID
```

```
_start_date = "11/04"
VALID
```

```
10_months = "..."
INVALID (Number Start)
```

```
enrolled students = 11k
INVALID (Space)
```

🧑‍🏫 Class Activity

Which ones will Python accept?

1. inceptez_ai
2. 2026_GenAI
3. True_AI_Course
4. info@inceptez

PART 3

The Building Blocks: Keywords & Variables

1. Keywords

Python's "locked" vocabulary. These are the **verbs** of the language.

- **Fixed Meaning:** Tells Python exactly what logic to run.
- **The Golden Rule:** Strictly forbidden for naming data (e.g., no variables named `if` or `for`).

2. Identifiers

The names you create for variables or projects.

✅ **DO:** Start with letters or `_`. Be case-sensitive (`Data` ≠ `data`).

❌ **DON'T:** Start with numbers, use spaces, or special characters like `@` or `%`.

3. Variables: Labeled Containers

```
course_duration = 10
VALID
```

```
_start_date = "11/04"
VALID
```

```
10_months = "..."
INVALID (Number Start)
```

```
enrolled students = 11k
INVALID (Space)
```

🧑‍🏫 Class Activity

Which ones will Python accept?

1. inceptez_ai
2. 2026_GenAI
3. True_AI_Course
4. info@inceptez

Ans: 1 & 3 are correct!

PART 4

Data Types — The Five Basic Flavors

1. Integer (int)

"Count Anything"

Whole numbers, exact quantities.

Ex: 11,000 students, 10 months.

2. Float (float)

"Measure Anything"

Precision numbers with decimals.

Ex: 4.8 rating, 85.0% hands-on.

3. Boolean (bool)

"Yes or No"

Simplest type for logic decisions.

Values: True / False

4. String (str)

"Letters & Text"

Wrapped in quotes to identify as text.

Ex: "inceptez", "2026"

5. Complex

"AI Research"

Advanced math with imaginary parts.

Ex: 3 + 5j

💡 Analogy: Ingredients vs. Recipe

Think of data types as categories of ingredients. You wouldn't use a "string" (text) to perform "math" (multiplication) any more than you'd use a spoon to cut an onion. Handling data correctly requires knowing its **flavor**.

👩‍🏫 Lab Activity

What's My Type?

```
1. "11/04/2026"
2. 12
3. 85.0
4. True
```

PART 4

Data Types — The Five Basic Flavors

1. Integer (int)

"Count Anything"

Whole numbers, exact quantities.

Ex: 11,000 students, 10 months.

2. Float (float)

"Measure Anything"

Precision numbers with decimals.

Ex: 4.8 rating, 85.0% hands-on.

3. Boolean (bool)

"Yes or No"

Simplest type for logic decisions.

Values: True / False

4. String (str)

"Letters & Text"

Wrapped in quotes to identify as text.

Ex: "inceptez", "2026"

5. Complex

"AI Research"

Advanced math with imaginary parts.

Ex: 3 + 5j

💡 Analogy: Ingredients vs. Recipe

Think of data types as categories of ingredients. You wouldn't use a "string" (text) to perform "math" (multiplication) any more than you'd use a spoon to cut an onion. Handling data correctly requires knowing its **flavor**.

👩‍🏫 Lab Activity

What's My Type?

```
1. "11/04/2026"
2. 12
3. 85.0
4. True
```

Instructor Ans: 1.Str, 2.Int, 3.Float, 4.Bool

PART 5

Operations — Putting Data to Work

1. Math Operations

For stats and model accuracy.

- **Addition (+):** 10 + 2
- **Subtraction (-):** 2026 - 2014
- **Multiplication (*):** 10 * 4
- **Division (/):** 11000 / 10
- **Power (**):** Stats vital

2. Text Operations

Manipulating strings (text).

- **Concatenation (+):** Glue text.
  `"AI" + " Lab"`
- **Repetition (*):** Repeat word.
  `"AI" * 3`
- **Case:** Upper/Lower.
  `"py".upper()`

3. Logic Operations: The Traffic Lights

Decides which way the data flows in your code.

Comparison (==, >, <)

Checking equality or size differences.

```
hands_on > 80 # True
```

🧑‍🏫 Lab: Predict Result

What happens in Python?

```
1. "Full Stack " + "Data Science"
2. 100 - 85
3. "Success! " * 5
4. 10 > 12
```

PART 5

Operations — Putting Data to Work

1. Math Operations

For stats and model accuracy.

- **Addition (+):** 10 + 2
- **Subtraction (-):** 2026 - 2014
- **Multiplication (*):** 10 * 4
- **Division (/):** 11000 / 10
- **Power (**):** Stats vital

2. Text Operations

Manipulating strings (text).

- **Concatenation (+):** Glue text.
  `"AI" + " Lab"`
- **Repetition (*):** Repeat word.
  `"AI" * 3`
- **Case:** Upper/Lower.
  `"py".upper()`

3. Logic Operations: The Traffic Lights

Decides which way the data flows in your code.

Comparison (==, >, <)

Checking equality or size differences.

```
hands_on > 80 # True
```

🧑‍🏫 Lab: Predict Result

What happens in Python?

```
1. "Full Stack " + "Data Science"
2. 100 - 85
3. "Success! " * 5
4. 10 > 12
```

Ans: 1.Concatenation, 2.15, 3.Repetition x5, 4.False

PART 6

Logical Operators — combining multiple conditions

AND Operator

Returns True if **both** statements are true.

```
x < 5 and x < 10
# True if x is 3
```

OR Operator

Returns True if **at least one** statement is true.

```
x < 5 or x > 20
# True if x is 25
```

NOT Operator

Reverse the result; returns False if the result is true.

```
not(x < 5 and x < 10)
# Flips the result
```

💡 Real-World Logic: Security Check

Think of logical operators as security filters. To access a "Pro" feature, a user might need a valid login **AND** an active subscription. However, for a "Guest" login, they might only need an email **OR** a social media link.

👨‍🏫 Practice Lab

Predict the output:

```
1. (10 > 5) and (5 == 5)
2. (10 == 9) or (1 > 0)
3. not(True or False)
```

PART 6

Logical Operators — combining multiple conditions

AND Operator

Returns True if **both** statements are true.

```
x < 5 and x < 10
# True if x is 3
```

OR Operator

Returns True if **at least one** statement is true.

```
x < 5 or x > 20
# True if x is 25
```

NOT Operator

Reverse the result; returns False if the result is true.

```
not(x < 5 and x < 10)
# Flips the result
```

💡 Real-World Logic: Security Check

Think of logical operators as security filters. To access a "Pro" feature, a user might need a valid login **AND** an active subscription. However, for a "Guest" login, they might only need an email **OR** a social media link.

👩‍🏫 Practice Lab

Predict the output:

```
1. (10 > 5) and (5 == 5)
2. (10 == 9) or (1 > 0)
3. not(True or False)
```

Ans: 1.True, 2.True, 3.False

PART 7

The Data Structure "DNA" Comparison

🟦 List

Type: [ ]

Heterogeneity: Supported

Index: Supported (0)

Mutability: Mutable via index
Mutability means ability to change the values of the data structure

🔒 Tuple

Type: ( )

Heterogeneity: Supported

Index: Supported

Mutability: Immutable

📖 Dictionary

Type: { k:v }

Heterogeneity: Supported

Index: Key-based

Mutability: Mutable

🧪 Set

Type: { }

Heterogeneity: Supported

Index: No Position

Mutability: No change via index

The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

PART 8

The Data Structure "DNA" Comparison

🟦 List

Type: [ ]

Heterogeneity: Supported

Index: Supported (0)

Mutability: Mutable

🔒 Tuple

Type: ( )

Heterogeneity: Supported

Index: Supported

Mutability: Immutable

📖 Dictionary

Type: { k:v }

Heterogeneity: Supported

Index: Key-based

Mutability: Mutable

🧪 Set

Type: { }

Heterogeneity: Supported

Index: No Position

Mutability: Mutable

The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

PART 8

Python List Operations

1. Basic Operations

- Access: `list[index]`
- Update: `list[i] = val`
- Delete: `del list[i]`
- Length: `len(list)`

2. Core Operations

- Add: `.append()`, `.insert()`
- Combine: `.extend()`
- Remove: `.pop()`, `.remove()`
- Sort: `.sort()`, `.reverse()`

3. Special Operations

- Slicing: `list[start:stop:step]`
- Comprehension: `[x for x in list]`
- Membership: `val in list`
- Repetition: `list * 3`

INSIGHT

Shifting the Perspective

Today isn't about becoming a **"Software Developer"**;
it's about becoming a **"Technology Enabler"**.